

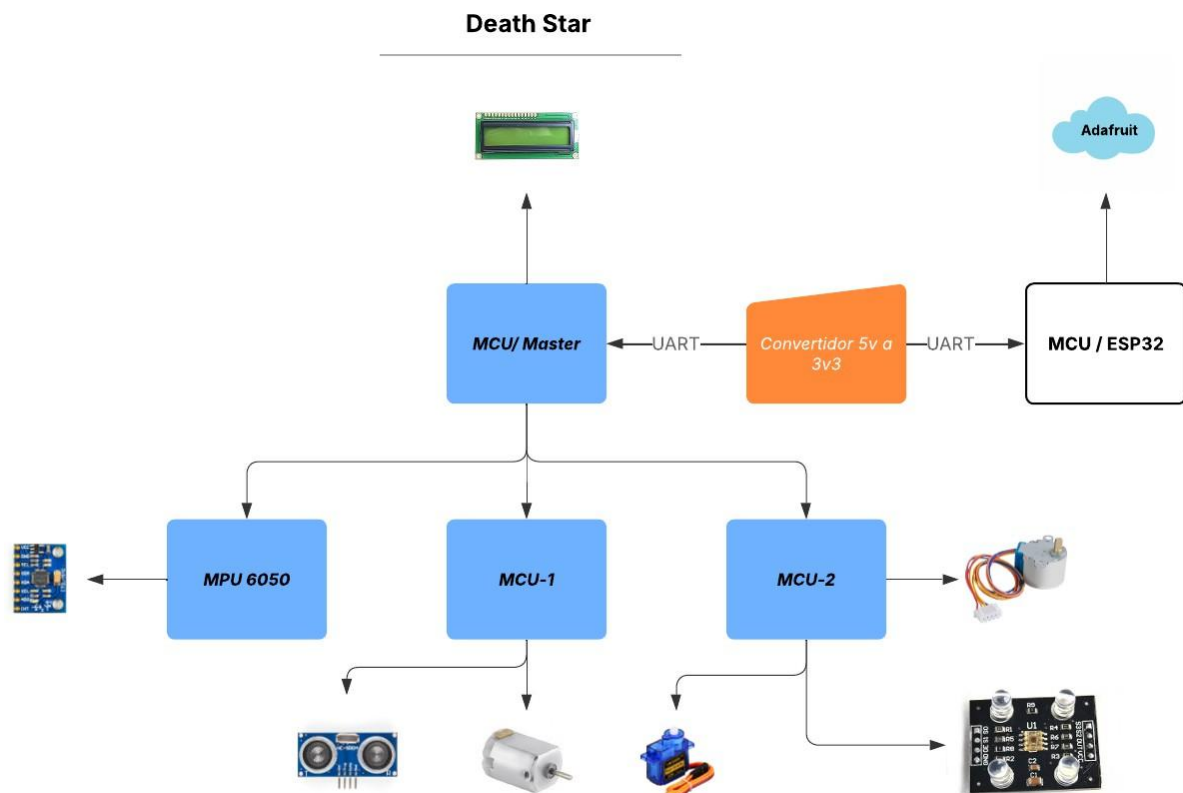
Proyecto 1: Monitoreo de red de sensores

Bryan Samuel Morales Paredes 23283
Ervin Gabriel Gomez García 231226

mor23283@uvg.edu.gt
gom231226@uvg.edu.gt

DEATH STAR

DIAGRAMA DE BLOQUES



Este proyecto simula un sistema embebido con el cual se pretende establecer un red de esclavos, los cuales a su vez controlan y actúan dependiendo de las decisiones que tome el maestro, es por ellos que cada uno de los esclavos cuenta con un sensor y motores, este consta de 5 módulos diferentes. El sistema funciona de la siguiente manera, el maestro constantemente está llamando a los esclavos por medio de comunicación I2C y captura los datos que se le envían, luego dependiendo del valor que esté presente será enviado para el esclavo correspondiente y este actúa dependiendo de estos valores capturados.

Tenemos el maestro, nuestro primer modulo, el cual se encarga de gestionar la comunicación I2C con cada uno de los esclavos, con el MPU siendo este un sensor

adaptado para comunicarse por I2C sin necesidad de un intermediario con los que siguen, el maestro inicializa su lectura, captura los datos necesarios, hace la lógica para devolver valores que se puedan trabajar, en este caso ángulos de giro en X y Y, estos datos son enviados al esclavo 2 (MCU-2), este los procesa y los envía como valores para mover el motor stepper y servo, al mismo tiempo que hace la lectura del sensor de color para luego enviarlo al maestro, este mismo procesa el dato para luego ser redireccionado al esclavo 1 (MCU-1) que dependiendo del valor del mismo dato, activa la secuencia de leds y por último el del laser de disparo. En el esclavo 1 (MCU-1) se tiene el sensor ultrasónico que detecta la distancia, este funciona de la siguiente manera, primero se procesa el valor del sensor para luego ser enviado al maestro éste procesa el dato, y dependiendo si éste rebasa el límite de cercanía, este envía un comando al esclavo 1 para activar el motor DC que simula un sistema de defensa. Por último el maestro se comunica con el ESP32 para enviar todos los datos de los sensores y actuadores (Motores) por medio de Serial, el ESP32 recibe los datos y los envía a la nube es decir a Adafruit para tener el estado actual de todo y también poder controlarlo desde este mismo.

video de funcionamiento: <https://youtu.be/vcDDvzLHe7Y>

Código destacable:

- *Estructura para enviar los comandos al esclavo 2 para hacer el movimiento del motor, mostrar y activar la secuencia de disparo.*

```
void enviar_comando()
{
    if(I2C_Master_Start())
    {
        if(I2C_Master_Write(slave2W))
        {
            // Primero enviar N o F
            if(bufferI2C <= 20){
                I2C_Master_Write('N');
                movimiento_motor = 1;
            }
            else{
                I2C_Master_Write('F');
                movimiento_motor = 0;
            }
        }

        //evaluar color
        if(color == 1){
            LCD_Set_Cursor(13,2);
            LCD_Write_String("R");
            I2C_Master_Write('X');
            autorizado = 0;
        }else if (color == 2 || disparo_IO == 1){
            LCD_Set_Cursor(13,2);
            LCD_Write_String("V");
            I2C_Master_Write('D');
            autorizado = 1;
        }else if (color == 3){
            LCD_Set_Cursor(13,2);
            LCD_Write_String("A");
        }
    }
}
```

```

        autorizado = 0;
    }

    }
    I2C_Master_Stop();
}
else
{
    I2C_Master_Stop();
    cadena_texto("Error: esclavo no responde\r\n");
}
}

```

- *Lógica para hacer el envío del ángulo al esclavo 1, enviando byte más alto y más bajo.*

```

void Enviar_angulos(int16_t anguloX, int16_t anguloY)
{
    I2C_Master_Start();

    if(I2C_Master_Write(slave1W))
    {
        I2C_Master_Write((uint8_t)(anguloX >> 8)); // Se envia el byte alto del
angulos x
        I2C_Master_Write((uint8_t)(anguloX & 0xFF)); // Se envia el byte bajo del
angulo x

        I2C_Master_Write((uint8_t)(anguloY >> 8)); // Se envia el byte alto del
angulo y
        I2C_Master_Write((uint8_t)(anguloY & 0xFF)); // Se envia el byte bajo del
angulo y

        I2C_Master_Stop();
    }
    else
    {
        // Si el esclavo no responde, se indica en el serial.
        I2C_Master_Stop();
        cadena_texto("Error: Esclavo no responde\r\n");
    }
}

```

- *Inicialización del MPU.*

```

void MPU6050_Init(void) {
    I2C_Master_Start(); I2C_Master_Write(MPU_W);
    I2C_Master_Write(PWR_MGMT_1); I2C_Master_Write(0x00);
    I2C_Master_Stop(); _delay_ms(10);
}

```

- *Uso de Interrupciones para la logica de lectura de sensor de color y movimiento del motor stepper.*

```
ISR(PCINT2_vect) {
//Interrupcion para un cambio en el pin D6, donde se encuentra el Out
    uint8_t current = (PIND & (1 << PIND6));

    if (current && !last_state) {
        pulsos_conteo++; // Solo flanco de subida
    }

    last_state = current;
}

ISR(TIMER0_COMPA_vect)
{
    // 1. Lógica del Stepper (Cada 10ms un paso)
    if (modo_stepper != 0) {
        if (modo_stepper == 1) paso_actual = (paso_actual + 1) % 4;
        else paso_actual = (paso_actual == 0) ? 3 : paso_actual - 1;

        PORTC &= ~0x0F; // Limpiar PC0-PC3
        PORTC |= (1 << paso_actual);
    } else {
        PORTC &= ~0x0F;
    }

    // 2. Lógica del Sensor de Color (Estado de tiempo no bloqueante)
    color_timer++;
    if (color_timer >= 10) { // Han pasado 100ms (10 * 10ms)
        color_timer = 0;

        switch(color_fase) {
            case 0: // Terminó lectura ROJO
                r_count = pulsos_conteo;
                PORTD |= (1<<PORTD5) | (1<<PORTD7); // S2=1, S3=1 (Filtro Verde)
                color_fase = 1;
                break;
            case 1: // Terminó lectura VERDE
                g_count = pulsos_conteo;
                PORTD &= ~(1<<PORTD5); PORTD |= (1<<PORTD7); // S2=0, S3=1
                // (Filtro Azul)
                color_fase = 2;
                break;
            case 2: // Terminó lectura AZUL
                b_count = pulsos_conteo;
                PORTD &= ~(1<<PORTD5); PORTD &= ~(1<<PORTD7); // S2=0,
                // S3=0 (Filtro Rojo)
                color_fase = 3; // Indicar al main que procese
                break;
        }
        pulsos_conteo = 0; // Reiniciar contador para el siguiente color
    }
}
```

- *Inicialización del ultrasónico y lógica de funcionamiento y devolución de datos*

```
void init_ultrasonic_icp(void)
{
    DDRC |= (1 << PORTC3);    // TRIG como salida
    DDRB &= ~(1 << DDB0);    // ICP1 (PB0 / D8) como entrada

    TCCR1A = 0;
    // Captura en flanco de subida inicialmente
    TCCR1B = (1 << CS11) | (1 << ICES1);

    TIFR1 |= (1 << ICF1);    // LIMPIAR FLAG
    // Habilitar interrupciones
    TIMSK1 |= (1 << ICIE1);

    TCNT1 = 0;
}

void trigger_ultrasonic(void)
{
    PORTC |= (1 << PORTC3);
    _delay_us(10);
    PORTC &= ~(1 << PORTC3);
}
```